

SQL Spaced Repetition Flashcard Deck

SQL Spaced Repetition Flashcard Deck

Companion to Chapter 1 — Operationalising the Evidence-Based Learning Framework

How to Use This Deck

Chapter 1 establishes that passive reading of SQL syntax creates only temporary familiarity — the forgetting curve erases unreinforced learning within days. This flashcard deck fights that decay by forcing active recall: you read the prompt, reconstruct the answer from memory, then check. Never skip the recall attempt, even when you feel uncertain.

Use the review intervals below, drawn from the spaced-repetition schedule described in Chapter 1:

Review Stage	Interval After Last Correct Answer	Goal
First exposure	Same day	Initial encoding
Stage 1	1 day later	Interrupt early forgetting
Stage 2	3 days later	Strengthen retrieval pathway
Stage 3	7 days later	Move toward long-term memory
Stage 4	14 days later	Consolidate durable skill
Stage 5	30 days later	Confirm lasting fluency

Mark each card Got it, Almost, or Review again. Cards marked "Review again" restart at Stage 1.

Deck 1 — Foundations (Progressive Layer 1)

Card 1

Q: What clause retrieves all columns from a table called employees?

A: SELECT * FROM employees;

Card 2

Q: How do you retrieve only the first_name and salary columns from employees?

A: SELECT first_name, salary FROM employees;

Card 3

Q: What keyword filters rows based on a condition?

A: WHERE — e.g., `SELECT * FROM employees WHERE salary > 50000;`

Card 4

Q: How do you sort results by salary from highest to lowest?

A: ORDER BY salary DESC;

Card 5

Q: What is the difference between WHERE and HAVING?

A: WHERE filters rows before aggregation; HAVING filters groups after a GROUP BY aggregation.

Deck 2 — Aggregation (Progressive Layer 2)

Card 6

Q: Name five SQL aggregate functions and what each computes.

A: COUNT() — number of rows; SUM() — total; AVG() — mean; MIN() — lowest value; MAX() — highest value.

Card 7

Q: Write a query that counts employees in each department.

A:

```
SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department;
```

Card 8

Q: What does COUNT(*) count that COUNT(column_name) does not?

A: COUNT(*) counts every row including NULLs; COUNT(column_name) counts only non-NULL values in that column.

Deck 3 — Joins (Progressive Layer 3)

Card 9

Q: What does an INNER JOIN return?

A: Only rows where the join condition matches in both tables.

Card 10

Q: Write a query joining employees and departments on department_id.

A:

```
SELECT e.first_name, d.department_name
FROM employees e
INNER JOIN departments d ON e.department_id = d.department_id;
```

Card 11

Q: When would you use a LEFT JOIN instead of an INNER JOIN?

A: When you need all rows from the left table regardless of whether a match exists in the right table — unmatched right-side columns return NULL.

Card 12

Q: What is a self-join and when is it useful?

A: A table joined to itself — useful for hierarchical data, such as finding each employee's manager when both are stored in the same employees table.

Deck 4 — Subqueries & CTEs (Progressive Layer 4)

Card 13

Q: What is a correlated subquery?

A: A subquery that references a column from the outer query and re-executes for each outer row.

Card 14

Q: Rewrite a subquery as a CTE. What keyword introduces a CTE?

A: WITH — e.g.:

```
WITH high_earners AS (  
    SELECT * FROM employees WHERE salary > 80000  
)  
SELECT * FROM high_earners;
```

Deck 5 — Concept Recall (All Layers)

Card 15

Q: Why does passive reading of SQL tutorials fail to build lasting skill, according to Chapter 1?

A: Because the forgetting curve erases unreinforced learning within days; without active recall and spaced repetition, recognition never becomes durable retrieval.

Card 16

Q: What is progressive skill layering and why does it prevent cognitive overload?

A: Building each new SQL concept on a fully consolidated foundation — joins only after SELECT/WHERE are solid — so working memory is never overwhelmed by simultaneous novelty.

Quick-Reference Card Counts by Layer

Layer	Topic	Cards
1	Foundations	1–5
2	Aggregation	6–8
3	Joins	9–12
4	Subqueries & CTEs	13–14
5	Concept Recall	15–16

Tip from Chapter 1: Never advance to the next layer until Stage 3 cards in the current layer are consistently marked "Got it." Compounding progress depends on the pyramid staying solid at every level.